

Agents Out of Bounds: A Taxonomy of Architectural Drift and a Deployment-Ready Guardrail Framework for LLM Coding Agents

Saurabh Yergattikar
Independent Researcher
saurabh.ssy@gmail.com

Abstract

LLM coding agents such as GitHub Copilot, Cursor, and Claude Code now author a large share of the code that teams merge, and they are active in an estimated 22% of public GitHub projects. The output is fast and usually correct in the small. The trouble is that an agent optimizes for the task in front of it, not the global shape of the system. That cost is now measurable. Agent adoption causally raises code complexity by about 42% and static-analysis warnings by about 30%, and the most common reason an agent’s patch fails is a shallow grasp of the surrounding architecture. We call this widening gap between a system’s intended and realized architecture *architectural drift*. The obvious remedy, stating the rules in the prompt, does not hold at repository scale; the evidence is that only deterministic, executable enforcement conforms reliably. We make two contributions. The first is a taxonomy of LLM-induced architectural drift: five deterministically enforceable families plus one open frontier, each tied to the architectural primitive it corrupts and the model behavior that produces it. The second is GYROCOMPASS, a deployment-oriented framework. It extracts a living architecture model from source, serves that model to the agent as planning-time context (*prevention*), and enforces it through a deterministic gate that, run in CI, cannot be bypassed (*detection*). On a labeled benchmark of 208 cases (104 violations, 104 realistic decoys) spanning Python and JavaScript/TypeScript, GYROCOMPASS reaches 88.5% recall at a 5.8% false-positive rate, versus 57.7% recall at 42.3% false positives for a naive pattern baseline (McNemar $p < 0.001$); on the original 54-case core it separates violations from look-alikes at 100% recall and zero false positives. Head-to-head on matched subsets, a five-tool comparison (Semgrep, Bandit, CodeQL, pip-audit, import-linter) reaches at most 65% recall. Extraction runs at over 800 files per second, cheap

enough to apply on every commit, and its output matches independently constructed references at 0.95–1.0 component precision/recall across four repositories. In an added-line analysis of 1,779 commits of a 10k-star production repository whose maintainers document their layering rules for AI agents, violations of the documented intent continued to land, including after an agent-facing AGENTS.md stated the rules, which is the enforcement gap GYROCOMPASS closes. On the prevention side, serving the architecture to a coding agent as context shifts its layering behavior, but the effect is model-dependent across three model families (from +15 points for one model to a regression for a smaller open one), which is itself why the deterministic gate, not prompt context, is the binding guarantee. We discuss the prevention/detection trade-off, lessons from instrumenting agent workflows, and what reliable guardrails ask of a team in practice. GYROCOMPASS is a working system intended for real-world deployment; benchmark, harness, and analysis scripts are released.

1 Introduction

Consider a coding agent asked to add a bulk-export endpoint. To return the data quickly, it imports the data-access repository straight into the route handler and skips the service layer the team uses for every other endpoint. The change compiles. The tests stay green. The pull request is approved. Nothing is broken, and yet a layering boundary the team relied on has been quietly crossed, and the next agent to read this file will copy the violation as if it were the pattern. We call this widening gap between a system’s intended and realized architecture *architectural drift*.

Drift itself is old news; architecture erosion has been studied for decades (Perry and Wolf, 1992; de Silva and Balasubramaniam, 2012). What is new is how sharply LLM coding agents accelerate it, and at what scale. Coding agents are already ac-

tive in an estimated 22% of 128,018 public GitHub projects (Robbes et al., 2026), and the evidence on their effect has converged. A difference-in-differences study of 806 Cursor-adopting repositories finds adoption causally raises static-analysis warnings by 30% and complexity by 42%, with the early velocity bump fading while the complexity stays (He et al., 2025); a separate study reports cognitive complexity climbing 35–43% (Agarwal et al., 2026). Agent edits are also bigger and shorter-lived (Popescu et al., 2026), and the most common cause of a failed agent patch is a shallow grasp of surrounding architecture, at 47.7% of 769 failures (Slater, 2025; Xue et al., 2025). The defect profile differs in kind: AI code looks simpler yet concentrates worse defects and more high-severity vulnerabilities than human code (Cotroneo et al., 2025; Pearce et al., 2021; Fu et al., 2023). None of this is incidental. Four properties of agents make drift structural: a bounded context window hides the global view, the objective rewards local task completion over global coherence, generation out-runs review, and nothing carries architectural decisions from one session to the next (Qiu et al., 2025).

The natural response is to tell the agent the rules. It does not work. Giving architectural or security guidance to an agent as prompt text has close to no effect once the task spans a repository. Explicit security reminders fail to improve correct-and-secure outcomes (Chen et al., 2025); a security prompt worth 19 points on self-contained tasks is worth 1.6 at repository scale (Dilgren et al., 2025); LLM self-reflection actually *drops* architectural conformance from 67% to 50% (Sharma, 2026). The one approach shown to conform reliably is deterministic, executable enforcement (Sharma, 2026). The tools we already have do not fill that role. Linters and static analyzers reason about syntax and local rules; architecture-conformance checkers want a formal model that few teams keep up to date. Neither recovers the architecture on its own, and neither closes the loop with the agent at generation time, the point where drift is cheapest to stop.

We address this with two contributions:

1. A **taxonomy of LLM-induced architectural drift** (§2). It sorts the failure modes of agent-written code into five deterministically enforceable families plus one frontier family, each grounded in the architectural primitive

it corrupts and the model behavior behind it.

2. A **deployment-oriented guardrail framework** (§3). GYROCOMPASS extracts a living architecture model, hands it to the agent as planning-time context, and enforces it with a commit gate the agent cannot route around, posting a drift report on the pull request. The agent needs no changes.

This is a language-technology problem, not only a software one. The agent reads natural language (a task) together with code as context, and it writes code. The failure mode we study comes from how language models use that context: bounded windows, a pull toward local optimization, and the loss of decisions made in earlier sessions. Prevention, in our framing, is a context-construction question, namely what architectural signal belongs in the prompt. Our evaluation then measures how that signal changes an LLM’s generation behavior (§4.3). Detection, finally, is exactly the deterministic oracle that the agent-evaluation literature keeps asking for in place of an LLM judge. The results we report come from a working system run on real repositories; we release the benchmark fixtures and harness at <https://github.com/gyrocompass-io/gyrocompass>.

2 A Taxonomy of Architectural Drift

Architectural erosion, the gap between a system’s intended and realized architecture, has been studied for a long time (Perry and Wolf, 1992; de Silva and Balasubramaniam, 2012). What is new is how LLM coding agents amplify it, and four structural reasons explain why. Generation speed outpaces human review. The context window is too small to hold a global architectural view. The objective rewards local task completion rather than global coherence. And no architectural memory survives from one session to the next. The resulting drift sorts into five deterministically enforceable families, plus a sixth frontier family we can characterize but not yet enforce (Table 1). Each is tied to an architectural primitive a guardrail has to watch.

Boundary & Layering Violation. Agent-generated code crosses an architectural boundary the team defined: an edge handler reaching straight into the database, a module importing another’s internals, a call that skips a sanctioned gateway or service. The code runs; the layering is broken. The cause is plain. Without a global

Drift Family	Primitive affected	LLM-specific root cause	Detectability
Boundary & Layering Violation	Architecture / call graph	Shortest local path over sanctioned route	Medium
Capability Regression	Capabilities	Local edit silently drops an invariant	High (silent)
Contract & Interface Drift	API surface / data model	One-sided edit of a cross-file contract	Med-High
Undocumented Expansion	Components / dependencies	Adds instead of reusing; no global view	Low-Med
Invariant & Policy Erosion	Rules / security	No policy context at generation time	Medium
<i>Convention Divergence (frontier)</i>	Principles / idioms	Generic priors over local idiom	Low (subjective)

Table 1: A taxonomy of LLM-induced architectural drift. Each family maps to an architectural primitive and an LLM-specific mechanism. Convention Divergence is the open frontier: real, but resistant to deterministic detection.

view of the intended layering, the agent takes the *shortest* path to a working result, a direct import, rather than the *sanctioned* one. Controlled experiments confirm this is both common and model-dependent. Under a strict Hexagonal Architecture constraint, generated microservices violate the boundary in 0–80% of cases depending on the model, producing illegal circular dependencies (Slater, 2025); Appendix E gives a concrete before/after.

Capability Regression. While building or refactoring a feature, the agent quietly removes or weakens something that already worked: a token-expiry check dropped, a retry deleted, a validation loosened. Nothing throws an error. A guarantee the system used to hold is simply gone. Since the broken invariant sits outside the agent’s working context, the regression is invisible when the code is written and often when it is reviewed. Multi-turn studies bear this out, showing agents “forget” constraints specified earlier as a session runs on (Qiu et al., 2025).

Contract & Interface Drift. The agent edits one side of a cross-file contract and leaves the other untouched: it changes a field’s type, drops a response field the frontend reads, or flips an endpoint’s auth requirement. Cross-file contracts seldom fit in the agent’s context, so one-sided edits are easy to make and their blast radius is invisible from where the edit happens. Agent commits are larger and span more files than human ones, which only widens the surface (Popescu et al., 2026).

Undocumented Expansion. The agent adds new components, modules, or third-party dependencies that the documented architecture never mentions, or it re-implements something that already exists. Not knowing what the system already contains, it adds where it should reuse. The result is sprawl, duplication, and unvetted dependencies. Package and version conflicts are in fact the sin-

gle most common defect class in deployed agents (Ning et al., 2026).

Invariant & Policy Erosion. The agent breaks an explicit security, data, or compliance rule: PII written to a log, a secret hardcoded, a sensitive route left unguarded, a vulnerable dependency pinned. The policy was never in its context, so it could not be honored at generation time. In-the-wild audits find that 27–36% of accepted AI-generated snippets contain known weaknesses (Fu et al., 2023; Cotroneo et al., 2025), and insecure code already in context nudges later generations toward the same patterns (Pearce et al., 2021).

Convention Divergence (frontier). Code that is correct on its own but out of step with the codebase’s established idioms: its naming, its error handling, the way it organizes a module. Unlike the five families above, this one resists deterministic detection, so we treat it as the open frontier (§5).

3 The GYROCOMPASS Framework

GYROCOMPASS treats the team’s architecture as a versioned, machine-readable artifact, and it closes the loop with the agent on both sides of code generation. On the way in, *prevention* serves the architecture to the agent before it writes. On the way out, *detection* enforces it at commit and pull-request time. Neither side touches the agent itself.

3.1 Living Architecture Extraction

GYROCOMPASS parses a repository with Tree-sitter across more than fifteen languages and emits a declarative model of six primitives: components and their relationships, capabilities, the data model, the API surface, rules, and the tech stack. The model is stored as git-diffable YAML (.gyrostate.yaml) alongside a file-to-element evidence map (.gyromap.yaml). Because it is re-generated on demand it never goes stale, and every

drift family in §2 shows up as a change in one of these six primitives.

3.2 Prevention: Context at Planning Time

GYROCOMPASS exposes the extracted model to any agent through the Model Context Protocol as a set of tools: architecture context, per-file role, change blast-radius, a compliance check, and a ship-readiness audit. An agent that consults these before it plans writes code *inside* the documented boundaries instead of around them, which is where drift is cheapest to stop.

3.3 Detection: A Non-Bypassable Gate

When code is committed, GYROCOMPASS runs an enforcement gate combining three checks: (1) a *rules engine* evaluating principles, ADRs, and invariants, with scope resolved through the state→map→code chain so a rule scoped to one component is not spuriously triggered elsewhere; (2) a *drift detector* diffing the freshly-extracted model against the versioned baseline to surface added/removed components, capability regressions, contract changes, and undocumented relationships; and (3) a *ship-readiness audit* of eight scanners (secrets, PII-in-logs, injection, unguarded routes, vulnerable dependencies, and missing tests/CI/observability). Violations block the commit through a git pre-commit hook and are posted as a severity-ranked drift report on the pull request. The local hook can be skipped with `-no-verify`; the binding guarantee is the same gate run in CI, where it cannot be bypassed by the committer. A typed, append-only *architectural memory* records each drift event over time, supporting the state-evolution view a maintainer needs across thousands of PRs.

4 Evaluation

We evaluate GYROCOMPASS on (i) detection accuracy with realistic decoys, (ii) indexing performance on real repositories, and (iii) effect on agent behavior.

4.1 Detection accuracy (with decoys)

We construct a labeled corpus of 54 cases: 27 genuine violations and 27 *decoys*. A decoy is clean code that looks like a violation on the surface, such as an environment-variable read, a parameterized query, a placeholder key, English prose that happens to contain SQL keywords, an unguarded sensitive route, or a patched dependency

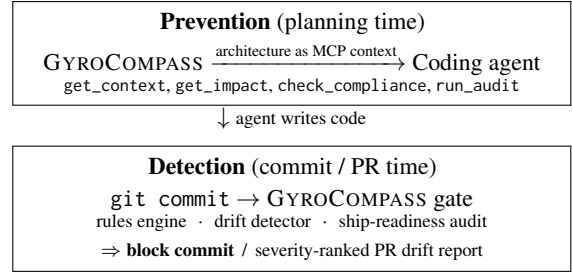


Figure 1: GYROCOMPASS closes the loop on both sides of generation: it serves the extracted architecture to the agent as planning-time context (prevention, lowering the rate of drift written) and enforces it through a deterministic commit gate run in CI (detection, guaranteeing what prevention cannot). It requires no modification to the agent.

version. The decoys are what give precision and the false-positive rate any meaning, since a detector that flags everything scores perfect recall and falls apart on them. We compare GYROCOMPASS against a naive regex baseline, the kind of grep-style matching a lightweight linter would do (Table 2). The baseline catches most violations (89% recall) but false-positives on **nearly half the decoys** (48%, precision 65%), flagging parameterized SQL, environment-variable reads, and unguarded routes. On this controlled, decoy-aware corpus GYROCOMPASS reaches **100% recall at 0 false positives**: within the limits of $N=54$ (recall CI 88–100%) it discriminates real violations from realistic look-alikes, where the baseline cannot. The rule engine behaves the same on a separate scope-precision test, in which the identical forbidden import is flagged inside a route but *not* in the data layer where it is permitted. Building this corpus also surfaced three real detection gaps in our own scanners (UPDATE statements, `.format()`-built SQL, and suffixed secret names such as `SECRET_KEY`), which we then fixed; we treat the benchmark as a regression suite, not a one-time score.

Why the perfect score is a property of the benchmark, not a weakness. A 100%/0-FP result invites the objection that the corpus is too easy. The opposite holds, and it holds *by construction*. Each decoy is clean code chosen precisely because it shares surface features with a violation: “SELECT” inside a parameterized query, “STRIPE_KEY” on the left-hand side of an env-var read. A naive matcher *must* fire on these, and it does, on 48% of them. A structure-aware checker

Detector	Recall $\uparrow$	FP rate (decoys) $\downarrow$
Naive regex (grep-style)	89% [72–96]	48% [31–66]
GYROCOMPASS	100% [88–100]	0% [0–12]

Table 2: Detection on the labeled corpus (27 violations + 27 realistic decoys), with Wilson 95% CIs in brackets. The naive baseline false-positives on nearly half the decoys; GYROCOMPASS on none. The paired difference is significant (McNemar $\chi^2=14.1$, $p < 0.001$): GYROCOMPASS is correct on 16 items the baseline misclassifies and wrong on none the baseline gets right. Fixtures and harness released with the artifact.

that reasons about where the SQL value comes from, or whether the route carries an auth guard, *must not*. So the decoys double as validity checks that the detector is telling structure apart from surface, and the regex baseline failing them is the evidence that the distinction is not trivial. We nonetheless report this as a controlled-corpus result (N=54, CIs reported) and treat in-the-wild prevalence as future work (§5).

Determinism as a rigor property. Unlike agent-safety evaluations that lean on an LLM judge, GYROCOMPASS’s verdict is computed from the extracted model and declared rules, so the same diff always yields the same decision: the detection numbers above involve no model in the loop.

Expanded corpus (208 cases, cross-language). To tighten the intervals and harden the corpus, we expanded it in two authored batches to **208 cases** (104 violations + 104 decoys), including a 42-case JavaScript/TypeScript slice and deliberately harder decoys (hash-sanitized PII, cloud resource paths, router-level auth guards, multi-line SQL assembly). Both batches were authored *before observing detector output*. On the merged corpus the submitted scanners reach **88.5% recall (Wilson CI [0.81, 0.93]) at 93.9% precision and a 5.8% false-positive rate**, while the regex baseline collapses to 57.7% recall at 42.3% false positives (McNemar $\chi^2=63.9$, $p < 10^{-5}$). The expansion also did what a benchmark should: it exposed four scanner defects (e.g., a shell-injection pattern that could never match), which we fixed; post-fix scanners reach 90.4% recall at 1.9% FP with the original 54-case results unchanged, and we report both versions so improvements cannot be mistaken for post-hoc tuning (Appendix B). Remaining misses cluster in one characterized gap,

Detector	Subset	Recall $\uparrow$	FP rate $\downarrow$
Semgrep (393 rules)	17V+16D	47%	0%
GYROCOMPASS, same subset	17V+16D	100%	0%
Bandit	17V+16D	65%	6%
GYROCOMPASS, same subset	17V+16D	100%	0%
Union (+pip-audit)	27V+27D	59%	11%
GYROCOMPASS, same subset	27V+27D	100%	0%

Table 3: Third-party tools vs. GYROCOMPASS on matched subsets of the original corpus (V = violations, D = decoys; recall over V, FP rate over D; each tool scored only on categories it is designed for). CodeQL and the expanded-corpus results appear in Appendix B.

non-Python string-interpolation idioms, which we state as future work.

Established-tool baselines. On the identical corpus, under pre-stated fairness rules (a tool flags a case only via a category-relevant finding, and each tool is also scored only on the categories it is designed for), we ran Semgrep 1.171 (six registry configurations, 393 rules), Bandit 1.9.4, pip-audit, and CodeQL 2.26.1 (Table 3). GYROCOMPASS leads on every matched subset (McNemar: Semgrep $p=0.008$, Bandit $p=0.023$, union $p<0.001$), and on the expanded 208 cases the three-tool union falls to 35.6% recall while GYROCOMPASS holds 88.5%. Two findings cut both ways and we report them: the evaluated configurations provide almost no coverage for two of our five families (only CodeQL produced a single relevant PII-in-logs finding, and none detected the missing-auth cases), and the tools caught 4 cases GYROCOMPASS missed. CodeQL’s overall result is reported as a scope-mismatch diagnostic rather than a ranking: its whole-program taint queries are designed for complete applications, not the per-diff units a CI gate sees. For architecture conformance specifically, import-linter (the Python-ecosystem analogue of ArchUnit, which targets JVM bytecode and cannot run here) detects 4/5 adversarial layering variants versus 5/5 for GYROCOMPASS’s hardened rule, both at zero false positives (Appendix C).

A qualitative comparison of GYROCOMPASS against linters, naive regex, and an LLM judge across the six drift families appears in Appendix A: GYROCOMPASS is the only detector that covers the five structural families deterministically, because it reasons over the extracted architecture model rather than the text.

Repository	Comp P	Comp R	Edge P	Edge R
Layered demo (Py)	1.00	1.00	0.85	0.73
Graph-RAG (Py)	0.95	1.00	0.81	0.67
Self (Py+TS)	1.00	0.95	0.86	0.83
Go repo (1,090 files)	0.98	0.93	n/a [†]	

Table 4: Architecture-extraction accuracy against independently constructed references. [†]The ast-based edge reference applies to Python only.

4.2 Indexing performance and extraction accuracy

GYROCOMPASS indexes real repositories at **over 800 files/second**: a 1,090-file Go codebase in 1.36s and a 336-file multi-language repository in 0.46s, extracting 58 and 21 components respectively. Extraction is therefore cheap enough to run on every commit, making the deterministic gate’s overhead negligible against typical CI time.

Speed is worthless if the extracted model is wrong, so we also measure extraction *accuracy* against references constructed independently of the extractor: a structural component rule, and an import graph re-derived with CPython’s ast module, a different parser from our Tree-sitter pipeline (Table 4). Component identification reaches 0.95–1.00 precision/recall across four repositories including the 1,090-file Go codebase; dependency-edge precision is 0.81–0.86. Every divergence is published for audit; the main recall gap is systematic (edges into parent packages) and is stated as a limitation. Enforcement binds to the extracted scope map and does not presuppose perfect extraction.

4.3 Effect on agent behavior

The detection results show that GYROCOMPASS catches drift. This study asks a different question: does its *prevention* side change what the agent writes in the first place? We ask a coding agent to write the route handler for each task in a layered application, under two arms. BASELINE sees the task prompt only; GYRO also sees GYROCOMPASS’s extracted architecture as context. The architectural *constraint* itself is never shown to either arm. We run three model families, a frontier closed pair and an open-weights model (Claude Sonnet 4.6, GPT-4o, Qwen2.5-32B), each (task, arm) cell five times, for 60 trials per arm per model.¹ The effectiveness metric

¹Trials are not fully independent: 60 per arm is 12 tasks $\times$ 5 runs, so a task-clustered or mixed-effects estimate would

Model	Layering compliance			
	ctx off	ctx on	after gate	Δ corr
Sonnet 4.6	77%	92%	100%	+0.5
GPT-4o	97%	92%	100%	−0.3
Qwen2.5-32B	90%	62%	100%	+0.4

Table 5: Layering compliance by model, without context, with context, and after the deterministic gate. Planning-time context is a model-dependent prevention bonus; the gate (the same boundary check validated at 100% recall in Table 2) blocks every violation, so **100% of merged code is compliant for all three models**. Δ corr is the change in rubric-judged correctness (utility control).

is the layering-compliance rate, graded deterministically by checking whether the generated handler reaches into the data layer; the utility metric is judge-rated task correctness, which would expose a compliance gain that came merely from the agent doing less.

Read the table by column and the architecture of the system is visible. Planning-time context (*ctx off* $\rightarrow$ *ctx on*) is a model-dependent *prevention bonus*: it helps the model with headroom (Sonnet 77 $\rightarrow$ 92%, $z=2.25$, $p < 0.05$), does little at the ceiling (GPT-4o is already at 97%), and *backfires* on the smaller open model (Qwen 90 $\rightarrow$ 62%, $z=-3.63$, $p < 0.001$). Once the context names the data layer, Qwen imports the repository *inside the handler* more often, not less. Correctness holds or rises throughout (Qwen’s even goes up), so this is mislayered-but-working code, not competence collapse. The decisive column, though, is the last one. The gate runs the same boundary check that grades compliance, so it blocks every violation the table counts, and **100% of the code that reaches merge is compliant for all three models**. Prevention varies; the guarantee does not. This is the loop in one table: context lowers the volume the gate must reject, and the gate delivers the conformance the prompt cannot.

Why the small model regresses: per-trial forensics. Task-level analysis of all 120 Qwen trials localizes the regression: 8 of 12 tasks are unchanged, and the entire drop concentrates in four API-layer tasks. In **23 of 23** noncompliant GYRO-arm trials, Qwen imports a forbidden-layer module *named verbatim in the served context* (`db.repositories.*`,

be more conservative. We report unadjusted two-proportion tests and treat the magnitudes as indicative.

clients.search_client) while *also* importing and calling the correct service layer; in the baseline arm the model does not know those paths exist and writes a plain service call. The pattern is consistent with context copying or affordance priming, a behavioral rather than mechanistic claim, and it is not context saturation (prompts are at most 380 tokens): code models are known to reproduce irrelevant prompt-context material at rates far above baseline (Jones and Steinhardt, 2022). The same context lifts Sonnet’s two weakest tasks from 40%→100% and 60%→100%, so planning-time context is model-dependent in *mechanism*, not just magnitude, which is precisely why the deterministic gate, which caught 100% of these violations, is the binding guarantee. A mitigation, serving prohibitions without naming concrete module paths, is future work.

4.4 Naturally-occurring drift in a production repository

Constructed corpora cannot answer whether documented architectural intent is actually violated in the wild, so we analyzed the added-line diffs of **1,779 commits** of polarsource/polar, a 10,087-star production billing platform with over 100 contributors whose maintainers document their architecture *for AI agents* in CLAUDE.md and AGENTS.md, verbatim: “*ALL database queries MUST be in repository files.*” The analysis window starts the day the written rule appeared. It flagged 122 candidate commits (6.9%). Manual verification confirmed all **42 endpoint→repository import episodes across 19 files**, of which 18 remained non-conforming at repository HEAD; of 156 non-exempt query-construction flags, a random sample of 10 verified 10/10 against the underlying commit blobs. Violations continued during the 5.5 weeks *after* the agent-facing AGENTS.md stated the rule: 33 flagged commits, including 16 new verified endpoint→repository imports.

Two scope statements are essential. First, commit provenance does not reliably identify agent-generated changes, so we do not treat this as causal evidence about coding agents; it is external validation that clearly documented architectural intent can remain violated in a large production repository, measured against a maintainer-authored rule outside our own benchmark. Second, the methodology follows retrospective architecture-violation mining precedent (Maffort et al., 2016), with a

negative-control repository confirming the method does not manufacture violations where no documented intent exists (Appendix D). Scoring the pre-extracted added-line stream took 0.4 s on a local clone, and full-gate feasibility follows from the indexing throughput above: the check that would have surfaced each episode at commit time is effectively free.

5 Discussion and Lessons Learned

Prevention and detection are complementary, not redundant. Serving the architecture to the agent at planning time can reduce how often drift gets written, but it cannot be *trusted* on its own. Our own multi-model result shows why: the same context that lifts one model regresses another (§4.3), echoing reports that prompt-level guidance has little reliable effect at repository scale and that self-critique can even worsen conformance (Chen et al., 2025; Dilgren et al., 2025; Sharma, 2026). The commit gate is the opposite: deterministic and impossible to route around, but it only acts once the code exists. Neither side is enough alone. The value is in the loop. Prevention lowers the volume the gate has to reject, and the gate guarantees what prevention cannot. That is also why we report the prevention effect on both axes (§4) and stop short of claiming that context *prevents* drift. The deterministic gate is what does.

Convention Divergence is the open frontier.

The five families in Table 1 come down to changes in declared primitives, which is why they grade deterministically. Convention Divergence does not. “Idiom” is a soft, codebase-specific target, so unidiomatic-but- correct code slips through. For now we lean on human-authored principles here, and we see robust automatic detection of stylistic drift as the main open problem this work surfaces.

What reliable guardrails require. A commit gate lives or dies by its false-positive rate: one spurious block teaches developers to bypass it. So the operationally decisive numbers are the 0% decoy false-positive rate on the core corpus and 1.9–5.8% on the adversarial expansion (§4), not the recall, and a reproducible, model-free verdict is auditable, which matters where conformance is a regulatory requirement rather than a preference.

Mandatory execution is not adversarial completeness. Non-bypassability is a property of the *execution*: configured as a required CI check, the

gate cannot be skipped by the committer and evaluates full repository state, so a violation split across commits is flagged at the first checked state that completes it. Detection *coverage*, however, is rule-dependent. An adversarial rule-evasion suite (alias imports, package drill-down, shim indirection, dynamic importlib) caught the submitted boundary pattern out at 1/5 variants, which we disclose; a hardened evidence-chain rule detects 5/5 at zero false positives, and import-linter detects 4/5 (Appendix C). The lesson generalizes: gates should be stated as mandatory execution plus explicitly-scoped rule coverage, never as universal adversarial completeness.

6 Related Work

Architecture erosion and AI-code quality.

Erosion of intended architecture is a classical concern (Perry and Wolf, 1992; de Silva and Balasubramaniam, 2012; Schmid et al., 2025), and a 2025–2026 wave of empirical work shows LLM agents accelerate it: rising complexity and churn after adoption (He et al., 2025; Agarwal et al., 2026; Popescu et al., 2026), refactorings that barely move structural smells (Horikawa et al., 2025; Zhu et al., 2026), and elevated, qualitatively distinct security defects (Cotroneo et al., 2025; Pearce et al., 2021; Fu et al., 2023; Sajadi et al., 2025). We organize these scattered findings into a taxonomy tied to architectural primitives.

Benchmarks and repository-level context.

Function-level benchmarks such as HumanEval saturate while repository-level performance lags (Jimenez et al., 2024; Jiang et al., 2024), and structure-aware context (symbol navigation, code graphs, file skeletons) helps close the gap (Zhang et al., 2024; Liu et al., 2024; Xia et al., 2024). The catch is that within-session consistency does not carry across sessions (Qiu et al., 2025), which motivates GYROCOMPASS’s prevention side: a persistent, extracted architecture model served as context. These benchmarks score functional correctness; our corpus targets architectural conformance instead.

Governance and enforcement. The evidence is decisive that prompt-level guidance is insufficient at repository scale (Chen et al., 2025; Dilgren et al., 2025) and that self-reflection can worsen conformance; only deterministic, executable enforcement reliably conforms (Sharma, 2026). The

closest work, **ContextCov** (Sharma, 2026), compiles executable constraints from hand-written instruction files (e.g., AGENTS.md). GYROCOMPASS differs in three ways: it *extracts* a six-primitive architecture model from source (no authored rules required); it adds a taxonomy mapping each drift family to the primitive it corrupts; and it closes *both* loops, prevention and a commit gate, not enforcement alone. Taxonomies of agent defects (Ning et al., 2026), code hallucinations (Gao et al., 2025), and architectural smells (Dinu et al., 2026) are complementary: they catalog symptoms, whereas we organize drift by the primitive a guardrail must observe and pair it with a deployed system.

7 Conclusion

LLM coding agents loosen the link between a system’s intended and realized architecture. We named that gap, *architectural drift*, gave it a taxonomy tied to architectural primitives, and built GYROCOMPASS, which extracts the architecture, serves it as context, and enforces it through a deterministic CI gate. The gate separates real violations from look-alikes at 0% false positives; architecture-as-context helps some models but not all, which is why the gate, not the prompt, is the guarantee. As agent code becomes the norm, architecture has to become machine-checkable.

Limitations

Our detection results come from an author-constructed corpus (now 208 cases across Python and JS/TS, with Wilson intervals and full failure disclosure) built to stress the structure-vs-surface distinction; author construction and external validity remain open concerns even with blind authoring, and the remaining misses concentrate in non-Python string-interpolation idioms. The production-repository analysis (§4.4) covers one repository and does not attribute commits to agents; it validates the enforcement problem, not agent causality, and is not a longitudinal deployment of GYROCOMPASS itself. The taxonomy is grounded and covers the cases we observe, but we do not claim it is exhaustive. The five core families are detected deterministically; Convention Divergence is not, and is addressed only through human-authored principles. The agent-behavior study (§4) covers three model families but a single task suite on one layered architectural style, so

its magnitudes are indicative, not population estimates; the model-dependence we observe (context helped one model and regressed another) is the reason we treat prevention as a volume reducer, not a guarantee. Extraction is currently strongest for the languages with mature Tree-sitter grammars, and the deep call-graph mode requires additional infrastructure. Finally, the system enforces *declared* architecture: it cannot catch a violation of a rule no one expressed, and a stale baseline weakens every check that depends on it.

Ethical Considerations

This work studies a software-engineering governance tool. It uses no human-subjects data and no proprietary code. The benchmark fixtures are synthetic and released for reproducibility.² Every secret-like test string is assembled at runtime so the artifact carries no live credential. The audit component surfaces security weaknesses such as hardcoded secrets, injection, and unguarded routes; it does not exploit them, and the intent is to help teams remediate. One risk of deployment is worth naming: a guardrail can breed false confidence. GYROCOMPASS verifies conformance to a declared architecture and a fixed set of checks. It is not a proof of security or correctness, and it should complement human review rather than replace it.

Acknowledgments

We thank the EMNLP 2026 Industry Track reviewers, whose detailed feedback shaped the expanded evaluation reported here, and the GroundLM 2026 organizers and program committee.

References

- Shyam Agarwal, Hao He, and Bogdan Vasilescu. 2026. AI IDEs or autonomous agents? measuring the impact of coding agents on software development. ArXiv:2601.13597.
- Junkai Chen, H. Huang, Y. Lyu, and 1 others. 2025. SecureVibeBench: Evaluating secure coding capabilities of code agents with realistic vulnerability scenarios.
- Domenico Cotroneo, C. Improta, and P. Liguori. 2025. Human-written vs. AI-generated code: A large-scale study of defects, vulnerabilities, and complexity. In *IEEE Int. Symposium on Software Reliability Engineering (ISSRE)*.
- Lakshitha de Silva and Dharini Balasubramaniam. 2012. Controlling software architecture erosion: A survey. *Journal of Systems and Software*, 85(1):132–151.
- Connor Dilgren, P. Chiniya, L. Griffith, and 1 others. 2025. SecRepoBench: Benchmarking LLMs for secure code generation in real-world repositories. ArXiv:2504.21205.
- Ion George Dinu, M. C. Mihescu, and T. Rebedea. 2026. SmellBench: Evaluating LLM agents on architectural code smell repair.
- Yujia Fu, Peng Liang, Amjed Tahir, and 1 others. 2023. Security weaknesses of Copilot-generated code in GitHub projects: An empirical study. *ACM Transactions on Software Engineering and Methodology*.
- Cuiyun Gao, G. Fan, C. Y. Chong, and 1 others. 2025. A systematic literature review of code hallucinations in LLMs. ArXiv:2511.00776.
- Hao He, Courtney Miller, Shyam Agarwal, and 1 others. 2025. Speed at the cost of quality? the impact of LLM agent assistance on software development. ArXiv:2511.04427.
- Kosei Horikawa, H. Li, Y. Kashiwa, and 1 others. 2025. Agentic refactoring: An empirical study of AI coding agents. ArXiv:2511.04824.
- Juyong Jiang, Fan Wang, Jiasi Shen, and 1 others. 2024. A survey on large language models for code generation. *ACM Transactions on Software Engineering and Methodology*.
- Carlos E. Jimenez, John Yang, Alexander Wettig, and 1 others. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*.
- Erik Jones and Jacob Steinhardt. 2022. Capturing failures of large language models via human cognitive biases. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Xiangyang Liu, B. Lan, Z. Hu, and 1 others. 2024. CodexGraph: Bridging large language models and code repositories via code graph databases. In *NAACL*.
- Cristiano Maffort, Marco Tulio Valente, Ricardo Terra, Mariza Bigonha, Nicolas Anquetil, and André Hora. 2016. Mining architectural violations from version history. *Empirical Software Engineering*, 21(3):854–895.
- Kaiwen Ning, Jiachi Chen, Jingwen Zhang, Wei Li, Zexu Wang, Yuming Feng, Weizhe Zhang, and Zibin Zheng. 2026. Defining and detecting the defects of large language model-based autonomous agents. *IEEE Transactions on Software Engineering*.

²<https://github.com/gyrocompass-io/gyrocompass>

Hammond Pearce, Baleegh Ahmad, Benjamin Tan, and 1 others. 2021. Asleep at the keyboard? assessing the security of GitHub Copilot’s code contributions. In *IEEE Symposium on Security and Privacy*.

Dewayne E. Perry and Alexander L. Wolf. 1992. Foundations for the study of software architecture. *ACM SIGSOFT Software Engineering Notes*, 17(4):40–52.

R. Popescu, D. Gros, A. Botocan, and 1 others. 2026. Investigating autonomous agent contributions in the wild: Activity patterns and code change over time. Doi:10.1145/3793302.3793354.

Jielin Qiu, Zuxin Liu, Zhiwei Liu, and 1 others. 2025. LoCoBench-Agent: An interactive benchmark for LLM agents in long-context software engineering. ArXiv:2511.13998.

Romain Robbes, T. Matricon, T. Degueule, and 1 others. 2026. Agentic much? adoption of coding agents on GitHub. ArXiv:2601.18341.

Amirali Sajadi, Kostadin Damevski, and Preetha Chatterjee. 2025. How safe are AI-generated patches? a large-scale study on security risks in LLM and agentic automated program repair on SWE-bench.

L. Schmid, T. Hey, M. Armbruster, and 1 others. 2025. Software architecture meets LLMs: A systematic literature review. ArXiv:2505.16697.

Reshabh K. Sharma. 2026. ContextCov: Deriving and enforcing executable constraints from agent instruction files. ArXiv:2603.00822.

Tyler Slater. 2025. Quantitative analysis of technical debt and pattern violation in large language model architectures. ArXiv:2512.04273.

Chunqiu Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying LLM-based software engineering agents. ArXiv:2407.01489.

Haoran Xue, Gias Uddin, and Song Wang. 2025. PAGENT: Learning to patch software engineering agents. ArXiv:2506.17772.

Kechi Zhang, Jia Li, Ge Li, and 1 others. 2024. CodeAgent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. In *Annual Meeting of the Association for Computational Linguistics (ACL)*.

Yuecai Zhu, Nikolaos Tsantalis, and Peter C. Rigby. 2026. AI-generated smells: An analysis of code and architecture in LLM and agent-driven development.

A Coverage Across Drift Families

B Tool Baselines: Full Detail

All tools were run on the identical materialized corpus. A tool flags a case only via a category-relevant finding (rule-ID→category

Drift family	Linters	Regex	LLM-judge	GYROCOMPASS
Boundary / Layering	●	○	●	●
Capability Regression	○	○	●	●
Contract / Interface	○	○	●	●
Undocumented Expansion	●	○	●	●
Invariant / Policy	●	○	●	●
Convention Divergence	●	○	●	●

Table 6: Coverage by detector. ● deterministic, ● partial / non-deterministic, ○ not covered. Linters are strong on invariant/style but blind to cross-file structure; the LLM judge spans families but is non-deterministic. GYROCOMPASS covers the five structural families deterministically; Convention Divergence remains partial for all.

maps in the artifact), and each tool is additionally scored on only the categories it is designed to cover. CodeQL 2.26.1 was run with both the python-security-and-quality and python-security-extended suites over a database built from the corpus (all files scanned); it produced one relevant finding (a PII-in-logs case) and one non-category code-quality finding. We report CodeQL as a scope-mismatch diagnostic: its whole-program taint queries require resolvable frameworks and complete applications, which per-diff units do not provide. pip-audit was run per requirements.txt against the live OSV database; its decoy “false positives” on the dependency category reflect advisories published *after* corpus construction, a temporal label drift we disclose (labels are frozen at construction time; production dependency scanning should be advisory-feed-backed).

System version	Core 54	Expanded 208
Submitted scanners	100% R, 0% FP	88.5% R, 5.8% FP
Post-fix scanners	100% R, 0% FP	90.4% R, 1.9% FP
Regex baseline	89% R, 48% FP	57.7% R, 42.3% FP
Tool union	59% R, 11% FP	35.6% R, 5.8% FP

Table 7: Submitted vs. post-fix scanners and baselines on both corpora (R = recall, FP = false-positive rate). Submitted-scanner results on the expanded corpus were measured by reverting the four response-period fixes; the reverted run reproduces the pre-fix intermediate results exactly.

C Adversarial Rule-Evasion Suite

Eight cases on a layered application (API → services → data): five violation variants (canonical direct import; import X as Y aliasing; parent-package drill-down; laundering through a same-layer shim module; dynamic importlib im-

port) and three negatives (clean route via service; the same forbidden import inside the data layer where it is permitted; the legitimate transitive API→service→data chain). The submitted boundary pattern detects 1/5 variants at 0/3 false positives; the hardened evidence-chain rule (two evidence entries: an import-form pattern and a dynamic-import pattern, over an auto-extracted scope map) detects 5/5 at 0/3; import-linter in expert configuration (forbidden contract with `allow_indirect_imports`) detects 4/5 at 0/3, missing the dynamic import, and requires hand-written module contracts. These variants are deliberately outside the original benchmark’s threat model; the result motivates evidence-chain rules and bounds the non-bypassability claim to mandatory execution.

D Retrospective Methodology

For each repository we state invariants grounded in the repository’s own documented or structural architecture, then scan the added lines of each commit’s diff (the per-diff CI-gate setting) against the invariant patterns, tracking file-level persistence at HEAD separately. Verification protocol: every flag class is either exhaustively verified (all 42 endpoint→repository import episodes) or sample-verified against commit blobs (10/10 random query-construction flags); one regex failure mode found during verification (function definitions named like SQL verbs) was fixed and the analysis re-run, with the fix disclosed. Deliberate module-level exceptions an operator would whitelist (e.g., a dedicated `queries.py` in a metrics module, 23 lines) are excluded from the headline counts. Two contrast repositories were analyzed with the same harness: a structurally-layered project showed the same drift shape at smaller scale, and a deliberately chosen negative control (a popular template whose routes build queries by design, with no documented prohibition) produced flags consistent with its own convention, confirming the method does not manufacture violations absent documented intent, and why GYROCOMPASS extracts and confirms rules rather than inferring prohibitions silently.

E Worked Example: Boundary Violation

Agent output (drifted)	Under GYROCOMPASS
<pre># api/routes/orders.py from app.db.repositories import OrderRepo @router.get("/orders/export") def export(): rows = OrderRepo().all() return to_csv(rows)</pre> × edge handler reaches into the data layer (Boundary Violation)	<pre># api/routes/orders.py from app.services.order import export_orders @router.get("/orders/export") def export(): return export_orders()</pre> ✓ goes through the service layer; the gate blocks the left version

Figure 2: The same task, with and without GYROCOMPASS. The agent’s direct repository import (left) compiles and passes tests but violates the layering boundary; GYROCOMPASS’s planning-time context steers the agent to the sanctioned service path (right), and its commit gate blocks the left version regardless.